PROPOSTA TÉCNICA / MVP

Sistema de Gestão de Horas e Apuramento de Faturação

Contexto: teste técnico Full-Stack · FONSAT

OBJETIVO DO DOCUMENTO

Definir um MVP demonstrável para registar, validar e apurar horas de trabalhadores por obra e por empresa cliente, com um cálculo rastreável dos valores de faturação.

Visão geral

Esta proposta descreve uma solução inicial de produto, arquitetura e implementação. Parte apenas do contexto transmitido verbalmente: funcionários, encarregados, administração/dono da empresa, empresas clientes e cálculo de horas para faturação por empresa/obra.

Estado dos requisitos

Confirmado na conversa	Hipóteses a validar
Necessidade de um MVP de gestão de horas	Quem regista horas e quem tem poderes para as alterar
Intervenientes: funcionário, encarregado, dono/administração e empresa da obra	Se a empresa cliente tem login próprio
Apuramento de horas para faturação por empresa/obra	Tabela de preços, regras de extras, fecho e exportação

Nota: o enunciado oficial ainda não foi partilhado. As opções de produto e tecnologia abaixo são recomendações, não requisitos confirmados.

1. Objetivos e âmbito do MVP

A aplicação deve permitir associar colaboradores a obras, registar horas trabalhadas, rever os registos e obter totais por obra/cliente num intervalo de datas. O foco é completar o fluxo de ponta a ponta antes de adicionar funcionalidades periféricas.

1.1 Perfis e permissões

Perfil	Pode fazer no MVP
Funcionário	Consultar as suas alocações; registar horas; consultar o estado dos seus registos.
Encarregado	Consultar os registos das obras sob a sua responsabilidade; aprovar ou rejeitar entradas.
Administração / dono	Gerir clientes, obras, colaboradores, alocações e preços; consultar apuramentos e fechos.
Empresa cliente	Entidade comercial associada à obra; acesso à plataforma apenas se o enunciado exigir.

1.2 Fluxo principal

1. Administração cria empresa cliente e obra, define o preço aplicável e aloca os funcionários.
2. Funcionário regista data, obra, hora de início, hora de fim e pausa (se aplicável).
3. O sistema valida datas, durações, permissões, alocação e potenciais sobreposições.
4. Encarregado consulta entradas pendentes e aprova ou rejeita.
5. Administração filtra período, cliente e obra e visualiza horas aprovadas/faturáveis e respetivo valor.
6. Se existir fecho, o sistema preserva uma referência estável do preço e dos registos incluídos.

REGRA CENTRAL

Horas registadas ≠ horas aprovadas ≠ horas faturáveis. O MVP deve tornar estes conceitos explícitos, mesmo que inicialmente as regras os façam coincidir.

1.3 Fora de âmbito, salvo pedido explícito

Processamento salarial; emissão fiscal de faturas; GPS/biometria; portal externo do cliente; workflows complexos de férias; microserviços; sincronização offline; faturação multi-moeda. Podem ser evoluções futuras, mas aumentam o risco do teste.

2. Modelo de dados proposto

Entidade	Campos-chave / razão de existir
User	id, name, email, passwordHash, role, active
ClientCompany	id, name, taxId (se exigido), active
Project	id, clientCompanyId, name, location, active
ProjectAssignment	id, employeeId, projectId, startsAt, endsAt
TimeEntry	id, employeeId, projectId, startsAt, endsAt, breakMinutes, status, version, createdAt
TimeEntryApproval	id, timeEntryId, supervisorId, action, reason, snapshot, createdAt
BillingRate	id, projectId, rateCents, validFrom, validTo; expandir por categoria se necessário
BillingPeriod	id, clientCompanyId, projectId, startsAt, endsAt, status, closedAt
BillingLine	id, periodId, timeEntryId, minutesBilled, rateCentsSnapshot, amountCents

2.1 Relações principais

ClientCompany 1 — N Project
Project 1 — N ProjectAssignment N — 1 User (funcionário)
Project 1 — N TimeEntry N — 1 User (funcionário)
TimeEntry 1 — N TimeEntryApproval
Project 1 — N BillingRate
BillingPeriod 1 — N BillingLine N — 1 TimeEntry

2.2 Regras de integridade

- TimeEntry: endsAt deve ser maior do que startsAt; breakMinutes não pode ser negativo ou exceder a duração.
- Não permitir que o mesmo funcionário tenha intervalos sobrepostos sem tratamento explícito da regra de negócio.
- Validar se o funcionário está alocado à obra na data do registo.
- Controlar acesso por perfil e, no caso do encarregado, também pela obra atribuída.
- Na aprovação, verificar se a versão do registo ainda é a mesma que o supervisor analisou.
- Usar IDs de operação/idempotency keys para evitar duplicação em repetição de pedidos HTTP.

Simplificação: no MVP, User pode representar funcionários e utilizadores administrativos com um campo role. Se houver encarregados por obra, usar uma relação própria de atribuição de supervisores.

3. Regras de horas e faturação

3.1 Cálculo de minutos trabalhados

Minutos trabalhados = (hora de fim − hora de início) − pausa. Guardar durações em minutos inteiros; converter para horas apenas na apresentação. Tratar explicitamente turnos que atravessam a meia-noite, se forem permitidos.

3.2 Cálculo do valor faturável

Exemplo meramente ilustrativo: um cliente paga 25,00 € por hora normal numa obra; apenas os registos aprovados e considerados faturáveis entram no apuramento.

Colaborador	Horas faturáveis	Preço / h	Subtotal
João	160 h	25,00 €	4.000,00 €
Pedro	152 h	25,00 €	3.800,00 €
Lucas	168 h	25,00 €	4.200,00 €
TOTAL	480 h		12.000,00 €

FÓRMULA BASE

Valor faturável (cêntimos) = soma, por linha, de minutos elegíveis × preço/hora em cêntimos ÷ 60, usando uma regra de arredondamento definida. Os totais mostrados acima são valores sem IVA.

3.3 Decisões críticas a validar

- O preço varia por obra, colaborador, função ou categoria profissional?
- Como são definidas horas normais, extraordinárias, noturnas, fins de semana e feriados?
- Os intervalos são sempre descontados? Existe arredondamento de tempos?
- Quem pode corrigir horas aprovadas? O que acontece após o fecho do período?
- O apuramento é apenas um relatório ou também precisa de exportação para faturação?

3.4 Consistência após o fecho

No fecho do período, guardar snapshots dos minutos e preços aplicados em BillingLine. Assim, alterar posteriormente um preço não reescreve faturações históricas. Impedir inclusão duplicada de uma mesma entrada no mesmo apuramento; correções de períodos fechados devem seguir um fluxo explícito.

4. Arquitetura sugerida

Para maximizar a entrega num prazo limitado, a opção inicial é um monólito modular com Next.js, TypeScript, Prisma e PostgreSQL. O frontend consome ações/rotas de servidor com autorização; regras de negócio ficam em serviços testáveis, separados dos componentes de interface.

```
Browser (funcionário / encarregado / administração)
        ↓
Next.js — páginas + componentes React
        ↓
API / server actions — autenticação + autorização
        ↓
Serviços de domínio — horas, aprovação, faturação
        ↓
Prisma ORM → PostgreSQL
```

4.1 Organização de pastas

```
src/
  app/              # rotas e páginas
  modules/
    auth/           # sessão e permissões
    companies/      # clientes
    projects/       # obras e alocações
    time-entries/   # lançamento e validação
    approvals/      # decisões do encarregado
    billing/        # apuramento por período
  components/
  lib/prisma.ts
prisma/schema.prisma
```

4.2 Endpoints exemplificativos

Método e rota	Ação
POST /api/time-entries	Criar registo de horas com validação/idempotência.
GET /api/time-entries?projectId=&from=&to=	Consultar horas conforme permissões.
POST /api/time-entries/:id/approve	Aprovar com controlo de versão.
POST /api/time-entries/:id/reject	Rejeitar com motivo.
GET /api/billing/preview?projectId=&from=&to=	Pré-visualizar total faturável da obra.

4.3 Segurança e fiabilidade

- Autenticação segura e verificação de permissões em todas as operações no servidor.
- Proteção contra alterações concorrentes (por versão do registo ou transação).
- Mensagens claras de validação; registo de quem aprovou/rejeitou e quando.
- Transações para ações que alterem múltiplas tabelas; migrations versionadas.
- Não confiar em employeeId ou role enviado pelo browser sem cruzar com a identidade autenticada.

5. Plano de execução e critérios de aceitação

Ordem	Entrega	Critério mínimo
1	Setup + schema + migrations	Aplicação corre, base de dados sobe e existem dados de demonstração.
2	Autenticação e acesso	Funcionário não vê dados de outros; encarregado só gere obras atribuídas.
3	Clientes, obras e alocações	Uma obra pertence a um cliente; funcionários podem ter várias alocações.
4	Registo de horas	Registo válido calcula minutos; entrada inválida é rejeitada.
5	Aprovação	Só supervisor autorizado aprova; conflito de versão é detetado.
6	Faturação	Filtro por cliente/obra/período e total de horas/valor consistente.
7	Qualidade da entrega	Testes essenciais, README, dados demo e breve vídeo/screenshots opcionais.

5.1 Testes que demonstram valor

- Unitário: 08:00–17:00 com 60 min de pausa = 480 minutos trabalhados.
- Unitário: total faturável calculado corretamente a partir de minutos e preço em cêntimos.
- Integração: entrada de horas de um utilizador não pode ser aprovada por supervisor sem acesso à obra.
- Integração: duas tentativas com a mesma chave de idempotência criam uma única operação.
- Integração: aprovação de versão desatualizada devolve conflito e não altera o registo.
- Fluxo ponta a ponta: lançar horas → aprovar → consultar valor por obra/período.

5.2 O que mostrar na apresentação

- Uma empresa cliente e uma obra já criadas, com funcionários alocados.
- O funcionário regista um turno; o encarregado encontra o pedido pendente.
- O encarregado aprova; o dashboard de faturação atualiza o total.
- Explicar duas decisões técnicas: separação entre registo/aprovação/faturação e integridade no banco.
- Mostrar testes automatizados e documentação de como executar o projeto.

PRIORIDADE DE ENTREGA

Um fluxo completo, demonstrável e testado vale mais do que muitos módulos parcialmente implementados. Começar simples e deixar pontos de evolução documentados.

6. Perguntas em aberto para o enunciado oficial

Tema	Pergunta para confirmar
Perfis	O dono administra uma única empresa? Existe acesso externo para clientes?
Registo	O próprio funcionário lança as horas ou o encarregado lança em nome dele?
Aprovação	O encarregado aprova por turno, dia, semana ou mês?
Preços	O preço/hora muda por colaborador, categoria, obra ou tipo de hora?
Contrato	A obra pode ter vários preços vigentes em períodos diferentes?
Cálculo	Existem feriados, extras, pausas, arredondamentos ou mínimos faturáveis?
Faturação	Precisamos de cálculo, relatório, exportação ou emissão de fatura?
Correções	Registos aprovados ou fechados podem ser corrigidos?
Entrega	Prazo, funcionalidades obrigatórias, testes, deploy, repositório e documentação?

7. Decisões técnicas para justificar na entrevista

- Monólito modular: reduz a complexidade operacional e permite evoluir para módulos separados, se um dia fizer sentido.
- PostgreSQL + Prisma: modelo relacional claro, migrations e operações transacionais.
- TimeEntry separado de BillingLine: o registo do trabalho não se confunde com o preço cobrado ao cliente.
- Preço e duração imutáveis no apuramento fechado: histórico previsível e auditável.
- Segurança baseada no papel e na obra: os dados do colaborador e da empresa cliente não devem ser expostos indevidamente.
- Testes de regras de negócio antes de otimizações: faturação correta é mais importante do que sofisticação prematura.

Versão 0.1 • Documento de preparação técnica • Atualizar após a leitura integral do desafio enviado pela FONSAT.